

REDES NEURONALES

Clasificación del tipo de pokemon

Cristian Camilo Cobo Mendoza - 202323571

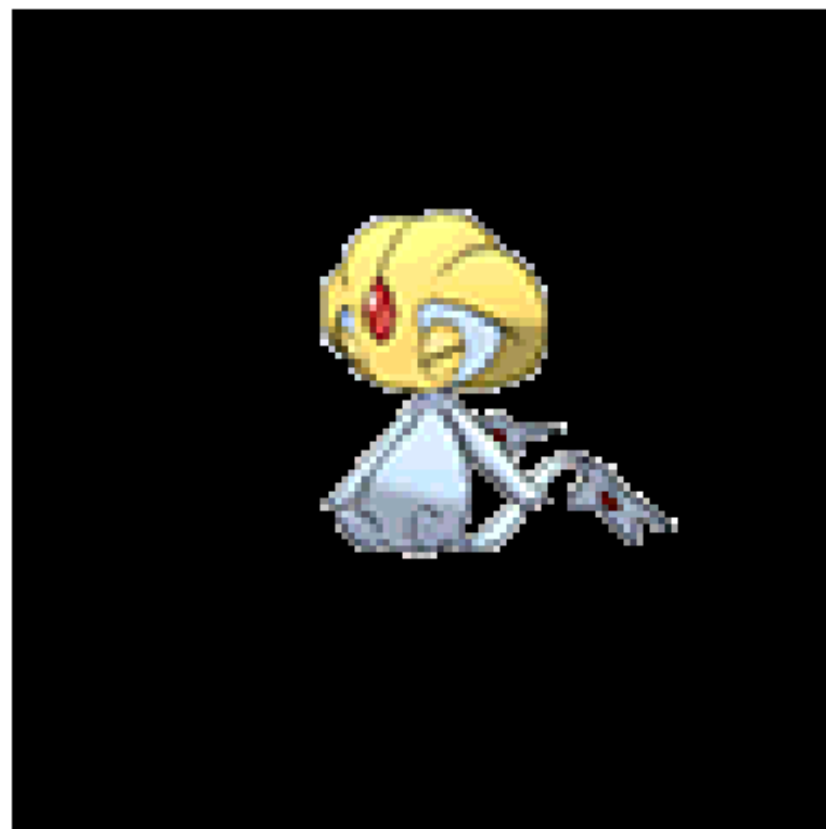
Emanuel Leal Arce - 202323555



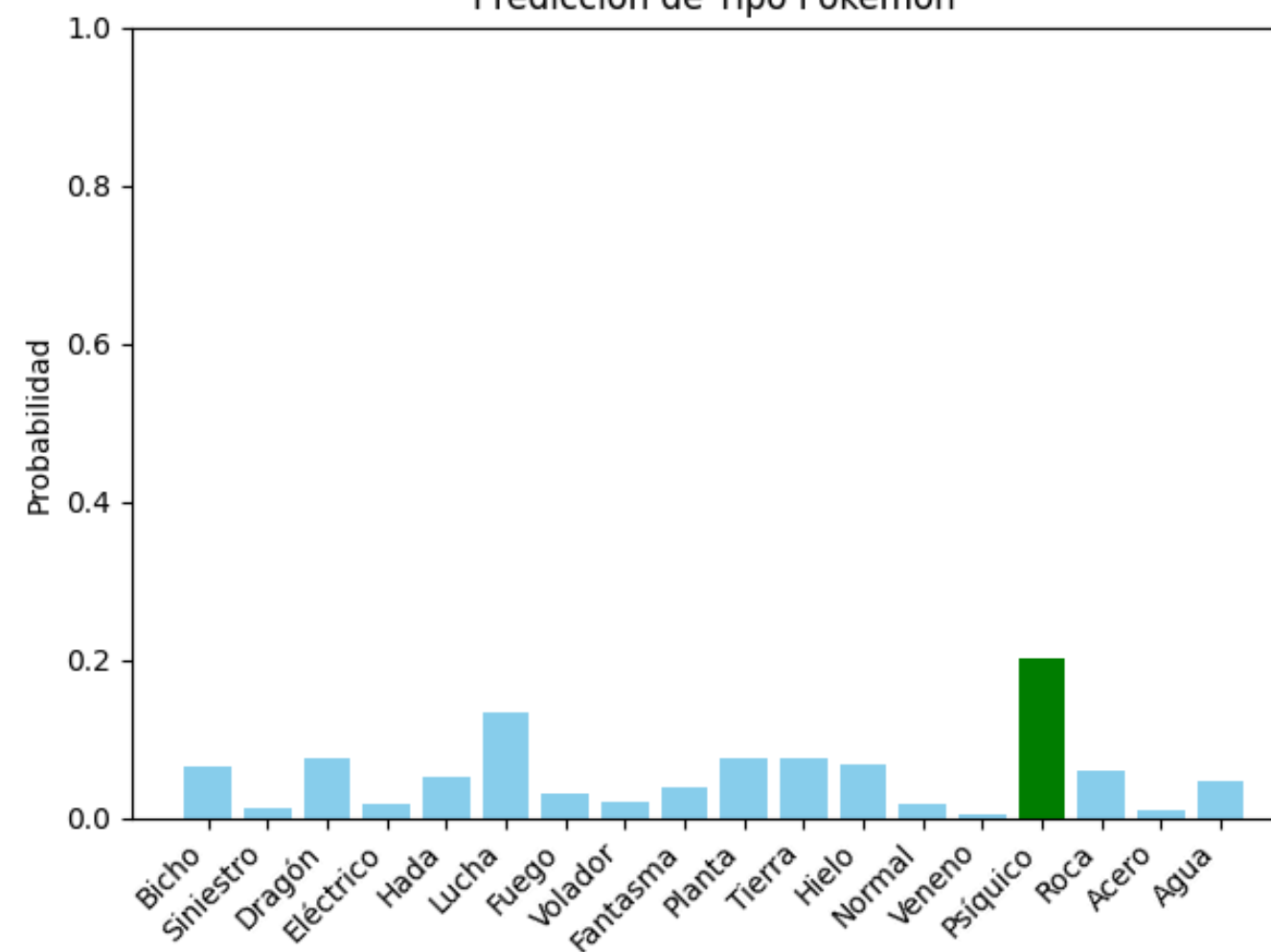
OBJETIVO DEL PROYECTO

Nuestro objetivo es desarrollar un modelo de red neuronal que pueda predecir el tipo principal de un Pokémon a partir de su imagen, aplicando técnicas de visión por computadora y aprendizaje profundo.”

Imagen: Uxie



Predicción de Tipo Pokémon



DATASET

El dataset utilizado en este proyecto fue extraído de un repositorio público en Kaggle y contiene:

- 809 imágenes de diferentes pokemones en formato png a color

DESAFÍOS DEL DATASET

- Desbalance de clases: Algunos tipos (como Water o Normal) aparecen mucho más que otros (Dragon, Fairy).
- Variabilidad visual: Algunos tipos comparten colores o formas, lo que complica la clasificación visual.



PREPARACIÓN DE DATOS

TRASFORMACIÓN DE IMAGENES

Se definen los preprocesamientos que se aplican a cada imagen:

- Se redimensiona a 128x128 píxeles
- Se normalizan los valores de píxeles al rango $[-1, 1]$

```
transform = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(
        (0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

CREACIÓN DEL DATASET Y DIVISION

- Se crea una instancia del PokemonDataset personalizado.
- Se divide el dataset en 80% entrenamiento y 20% validación usando random_split.

```
full_dataset = PokemonDataset(
    df, image_dir, transform=transform)
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(
    full_dataset, [train_size, val_size])
```



MODELO DE RED UTILIZADO

Utiliza:

- 3 capas convolucionales + ReLU + max pooling.
- 2 capas lineales finales.

Ventajas:

- Captura patrones espaciales y jerárquicos de la imagen.
- Requiere muchos menos parámetros al reducir dimensionalidad progresivamente.
- Generaliza mucho mejor en tareas de visión por computadora.



```
# --- Red convolucional ---
class CNNClassifier(nn.Module):
    def __init__(self, output_size):
        super().__init__()
        self.conv_layers = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc_layers = nn.Sequential(
            nn.Linear(128 * 16 * 16, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, output_size)
        )
```


ENTRENAMIENTO DEL MODELO

El proceso de entrenamiento de la red consiste en:

D

- Definición de etiquetas
- Conteo de clases(Cuantos tipos de pokemon existen)
- Definición de Pesos
- Configuración de la Red con dichos Pesos

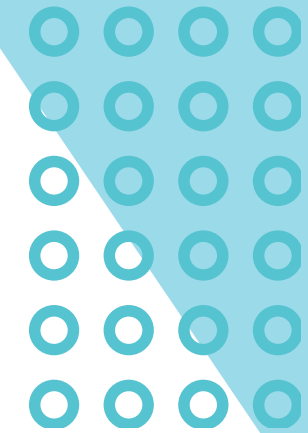
```
# --- Pérdida ponderada ---
labels = df["type"].map(type_to_idx)
class_counts = Counter(labels)
total = max(class_counts.values())
weights = [total / class_counts[i] for i in range(len(type_to_idx))]
weights = torch.tensor(weights, dtype=torch.float32)
criterion = nn.CrossEntropyLoss(weight=weights)
```

ENTRENAMIENTO DEL MODELO

Una vez definido lo anterior, establecemos el learning rate (Tasa de aprendizaje) en 0.003 y se utiliza Adam para el optimizer, a su vez definimos la cantidad de epochs (épocas):

¿Que son las épocas?: Las épocas definen cuantas veces la red pasara por la totalidad del dataset para definir y configurar dichos los pesos correctos, entre mas épocas, es posible que la red sea mas precisa, pero a su vez puede llevar a sobreajustarse, donde la red va en círculos y es contraproducente en terminos de tiempo y recursos del computador.

```
optimizer = optim.Adam(model.parameters(), lr=0.003)
epochs = 30
```



ENTRENAMIENTO DEL MODELO

Si no existe modelo de entrenamiento, se hara un bucle anidado en base a las épocas definidas y ajusta el modelo en modo entrenamiento:

```
for epoch in range(epochs):  
    running_loss = 0  
    model.train()
```

Hay un bucle anidado recorriendo todo el dataset, definiendo el tipo y la imagen del pokemon, y posteriormente prediciendolo, una vez hecha la predicción se calcula la perdida y precisión:

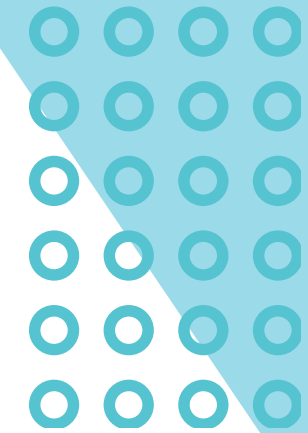
```
for images, labels in train_loader:  
    optimizer.zero_grad()  
    outputs = model(images)  
    loss = criterion(outputs, labels)  
    loss.backward()  
    optimizer.step()  
    running_loss += loss.item()  
  
# Validación  
val_loss = 0  
val_accuracy = 0
```


ENTRENAMIENTO DEL MODELO

Una vez se realiza la predicción, la red entra en modo evaluación, ahí calculando la precisión de esta y haciendo los ajustes necesarios

```
model.eval()
with torch.no_grad():
    for images, labels in val_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        val_loss += loss.item()
        ps = F.softmax(outputs, dim=1)
        top_p, top_class = ps.topk(1, dim=1)
        equals = top_class.squeeze() == labels
        val_accuracy += torch.mean(equals.type(torch.FloatTensor)).item()

print(f"Época {epoch+1}/{epochs}.. "
      f"Loss Entrenamiento: {running_loss:.3f}.. "
      f"Loss Validación: {val_loss:.3f}.. "
      f"Precisión Validación: {val_accuracy / len(val_loader):.3f}")
```



RESULTADOS

El modelo entrenado logró una precisión de validación promedio de aproximadamente un 90% durante la evaluación individual, el modelo predice correctamente el tipo de la mayoría de los Pokémon, especialmente aquellos con características visuales bien definidas.

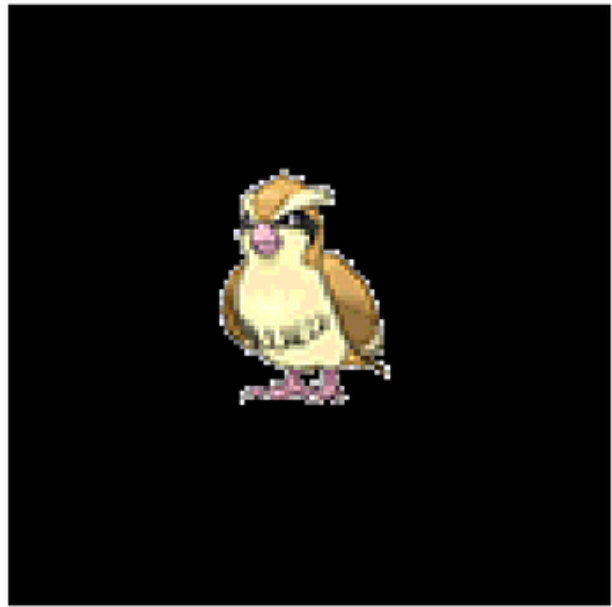
Los errores de predicción suelen concentrarse en Pokémon con tipos visualmente similares o imágenes poco distintivas.

El valor de pérdida (loss) en validación se mantuvo por debajo de 1.0 en la mayoría de los casos exitosos, mientras que valores superiores a 1.5 estuvieron relacionados con errores. El modelo final fue guardado para permitir futuras pruebas sin reentrenamiento.

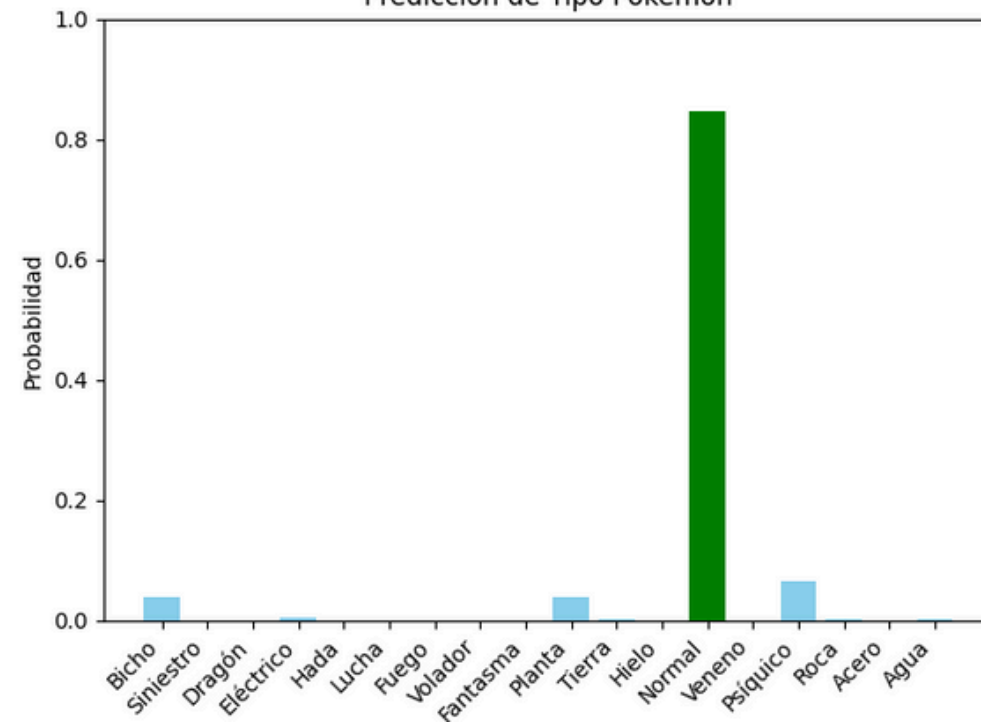


RESULTADOS

pidgey



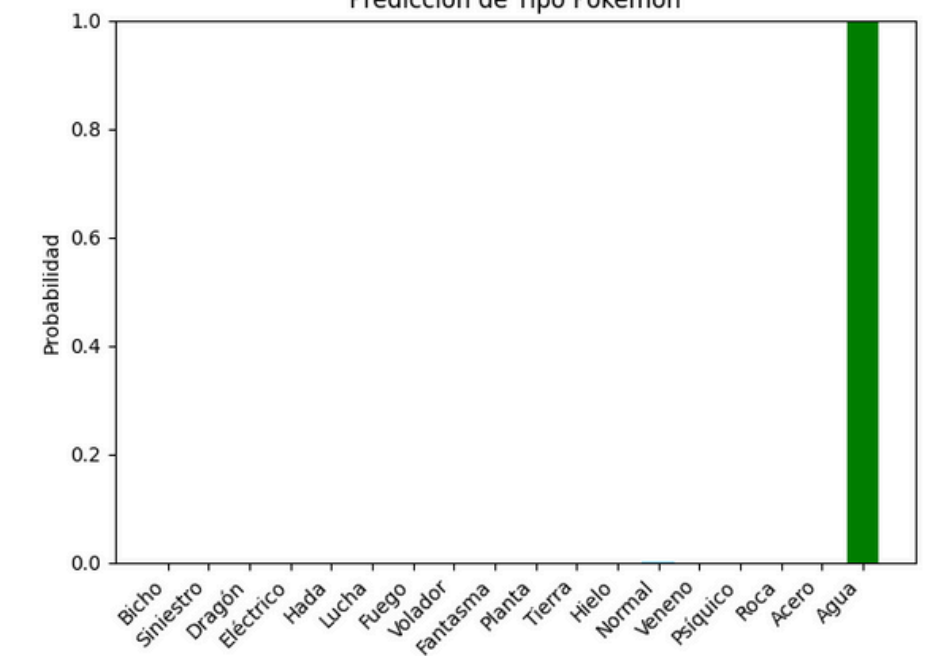
Predicción de Tipo Pokémon



ducklett



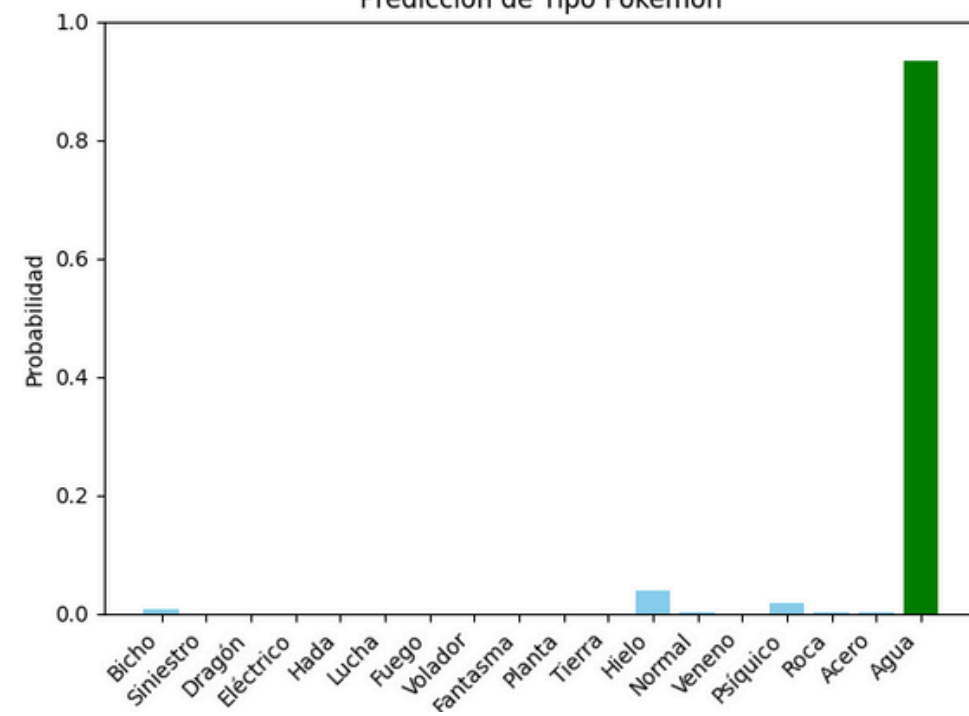
Predicción de Tipo Pokémon



qwilfish



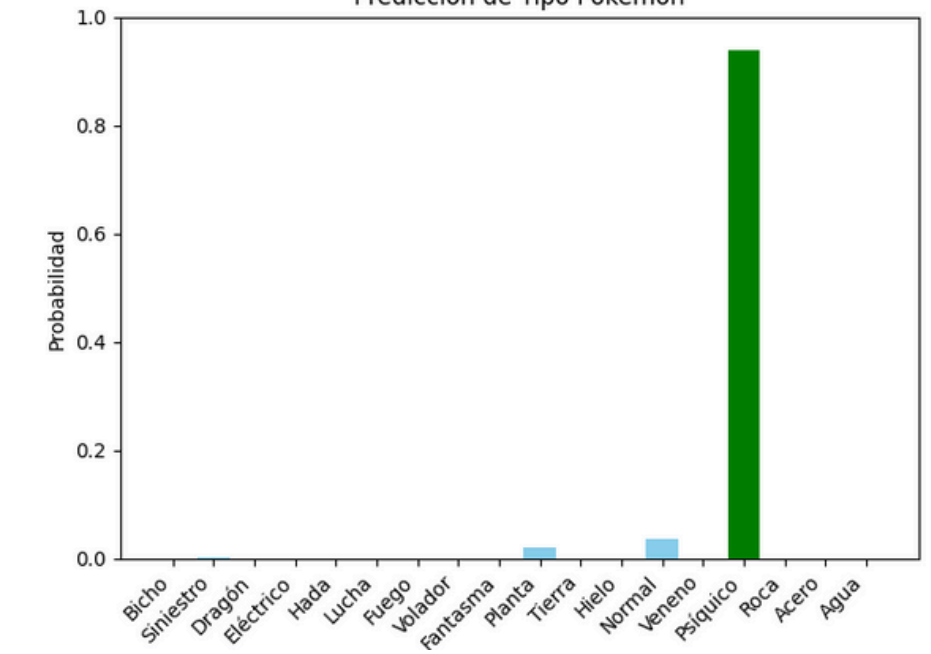
Predicción de Tipo Pokémon



spoink

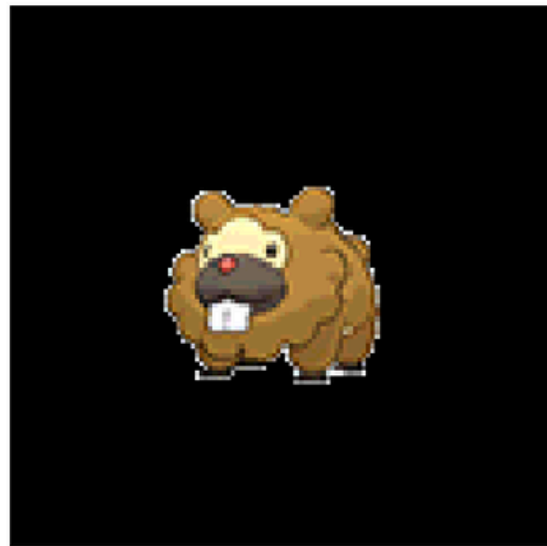


Predicción de Tipo Pokémon

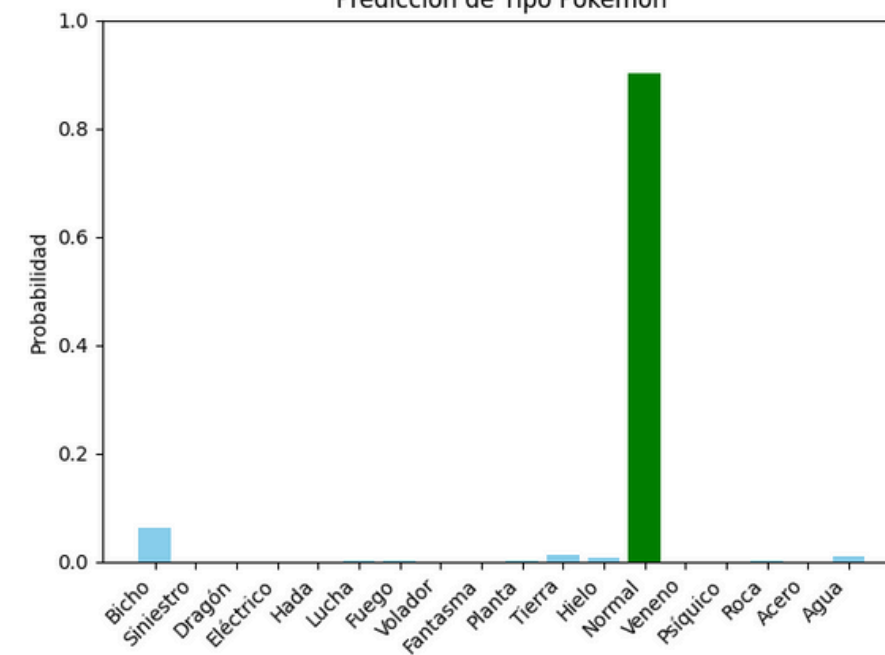


RESULTADOS

bidoof



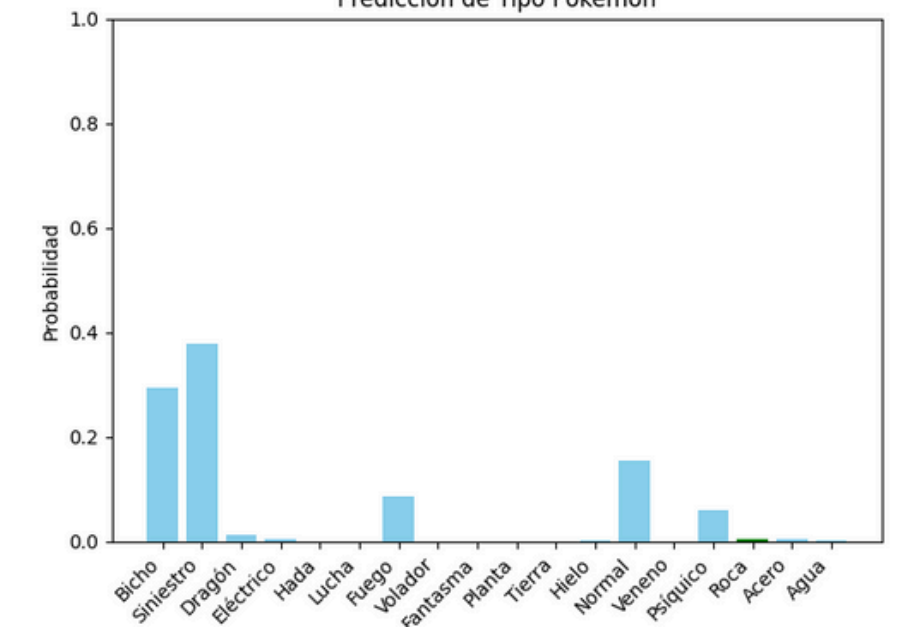
Predicción de Tipo Pokémon



binacle



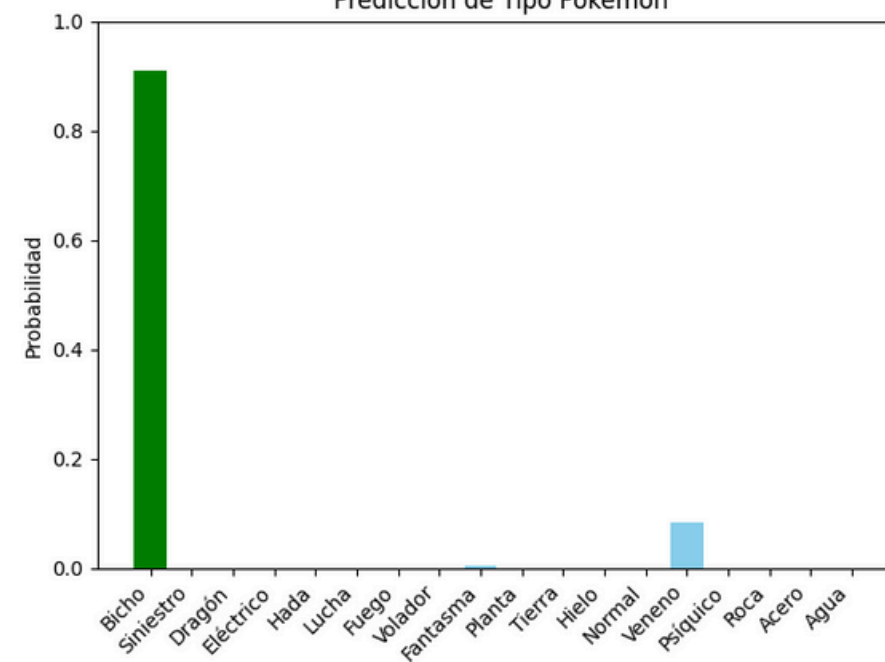
Predicción de Tipo Pokémon



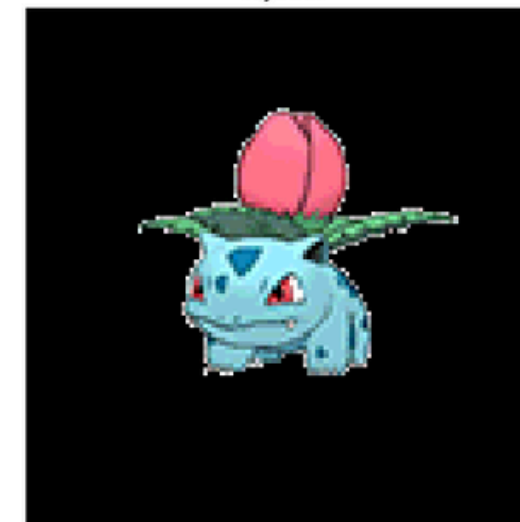
surskit



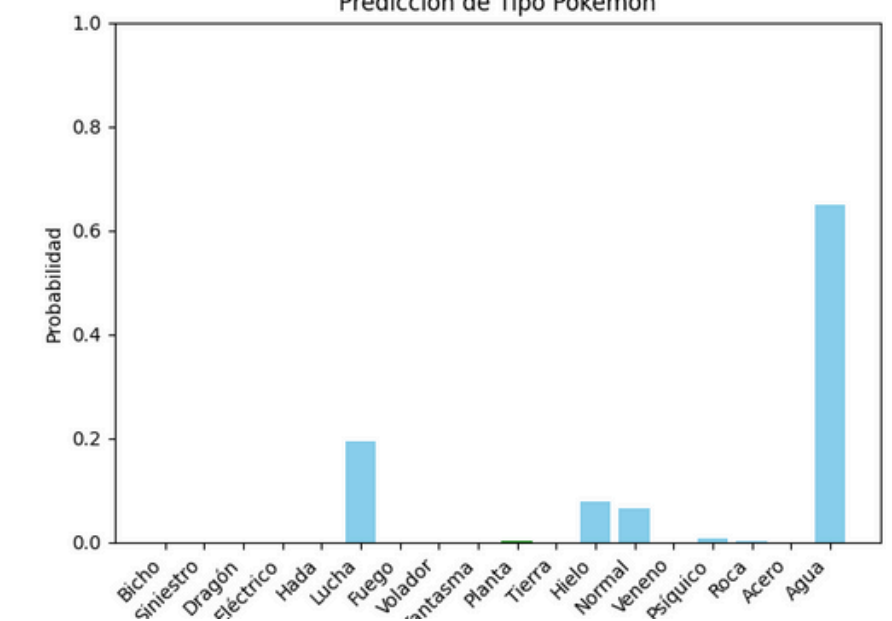
Predicción de Tipo Pokémon



ivysaur



Predicción de Tipo Pokémon



CONCLUSIÓN

Se demostró que es posible utilizar una red neuronal convolucional (CNN) para clasificar el tipo principal de un Pokémon a partir de su imagen.

Se llevó a cabo un proceso completo que incluyó la preparación del dataset, el diseño del modelo, el entrenamiento supervisado y la validación del rendimiento.

El modelo logró una precisión aceptable, destacando su capacidad para generalizar en la mayoría de los casos. Sin embargo, mostró algunas fallas ante imágenes ambiguas o de tipos visualmente similares.

Con un mayor volumen de datos y una variedad más amplia de imágenes, se podría mejorar significativamente la capacidad de generalización del modelo, reduciendo errores y fortaleciendo el entrenamiento.



**GRACIAS POR SU
ATENCIÓN**

